```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

#include "linked_list.h"

node_t* new_node(int n) {
    node_t* out = malloc(sizeof(node_t));
    out->next = NULL;
    out->val = n;
    return out;
}

list_t* create_empty_list() {
    list_t* linked = malloc(sizeof(list_t));
    linked->head = NULL;
    linked->tail = NULL;
    linked->length = 0;
    return linked;
}

list_t* create_example(int n) {
    list_t* linked = create_empty_list();
    for (int i = 0; i < n; i++) {
        append(linked, i);
    }
    return linked;
}

list_t* create_single(int n) {
    list_t* linked = create_empty_list();
    push(linked, n);
    return linked;
}

list_t* deep_copy(list_t* l) {
    list_t* copy = create_empty_list();
    node_t* to_append;
    for (node_t* c = l->head; c != NULL; c = c->next) {
        append(copy, c->val);
    }

    return copy;
}

bool is_empty(list_t* linked) {
    return (linked->length == 0);
}

void show_list(list_t* linked) {
    printf("TÊTE: %d\n", linked->head->val);
    printf("QUEUE: %d\n", linked->tail->val);
    printf("LONGUEUR: %d\n", linked->length);

    node_t* curr = linked->head;
    while (curr != NULL) {
        printf("%d -> ", curr->val);
        curr = curr->next;
    }
```

```c
    printf("FIN\n");
}

void show_values(list_t* linked) {
    for (node_t* c = linked->head; c != NULL; c = c->next) {
        printf("%d -> ", c->val);
    }
    printf("T\n");
}

void push(list_t* linked, int n) {
    node_t* new_head = malloc(sizeof(node_t));
    new_head->val = n;

    node_t* old_head = linked->head;
    new_head->next = old_head;
    linked->head = new_head;
    linked->length += 1;

    if (linked->length == 1) {
        linked->tail = new_head;
    }
}

void append(list_t* linked, int n) {
    assert(linked != NULL);
    if (linked->length == 0) {
        node_t* new_tail = new_node(n);
        linked->tail = new_tail;
        linked->head = new_tail;
        linked->length++;
    } else {
        node_t* new_tail = malloc(sizeof(node_t));
        new_tail->val = n;
        new_tail->next = NULL;

        node_t* old_tail = linked->tail;
        old_tail->next = new_tail;
        linked->tail = new_tail;
        linked->length += 1;
    }
}

void insert(list_t* linked, int index, int x) {
    assert(index <= linked->length);
    if (index == 0) {
        push(linked, x);
    }
    if (index == linked->length) {
        append(linked, x);
    }
    else {
        node_t* new_node = malloc(sizeof(node_t));
        new_node->val = x;

        node_t* curr = linked->head;
        int i = 0;
        while (i < index-1) {
            curr = curr->next;
            i += 1;
```

```
        }
        new_node->next = curr->next;
        curr->next = new_node;

        linked->length += 1;
    }
}

int pop(list_t* linked) {
    assert(is_empty(linked) == false);
    node_t* new_head = linked->head->next;
    node_t* old_head = linked->head;
    int val = old_head->val;
    linked->head = new_head;
    free(old_head);
    return val;
}

node_t* free_get_next(node_t* node) {
    assert(node != NULL);
    node_t* next = node->next;
    free(node);
    return next;
}

void free_list(list_t* linked) {
    assert(linked != NULL);
    node_t* curr = linked->head;
    while (curr != NULL) {
        curr = free_get_next(curr);
    }
    free(linked);
}

```